BitMamba-CV: Co-Projeto Hardware-Algoritmo de Modelos Mamba-2 Ternários com Discretização CPoT para FPGA de Borda

André Goveia[1]

Programa de Pós-Graduação – Aplicações de Alto Desempenho
andre@instituicao.edu.br

Abstract. *State space models (Mamba-2) keep a constant-size recurrent state, which makes them attractive for low-cost FPGAs, but their decay factor $\bar{A}_t$ lives close to 1 and is fragile under shift-only arithmetic. We present **BitMamba-CV**, a co-design of 1.58-bit ternary Mamba-2 and a multiplier-free selective-scan datapath for the Intel Cyclone V. We (i) train ternary projections in the Hadamard domain (`H-BitLinear`) together with an integer-compatible scan (static power-of-two scales, power-of-two Δ_t); (ii) propose the* Complementary Power-of-Two *(CPoT, $\{2^{-k}, 1-2^{-k}\}$) discretisation of $\bar{A}_t$, implemented with one shift and one subtraction; and (iii) show that CPoT is only correct with enough state precision: a 16-bit truncating state freezes long-memory channels (integer datapath SQNR 37,0 dB), while a 32-bit state with 10 guard bits and rounding reaches 99,0 dB, matching a Q1.15 multiplier (99,3 dB) with fewer DSP blocks; on a delayed-recall task ($L = 256$) the same trained model drops from 100,0% to 85,6% with the 16-bit state. Ternary models with the integer scan keep FP32 accuracy on ECG5000 and on recall with $L = 256$, but ternary training does not converge for most seeds at $L = 1024$. All RTL modules are verified bit-exactly against an integer golden model (9.384 vectors, 0 mismatches) and characterised with open-source synthesis for Cyclone V. Timing closure, on-board latency and power are left to the hardware phase and are not claimed here.*

Resumo. *Modelos de espaço de estados (Mamba-2) mantêm um estado recorrente de tamanho constante, o que os torna atrativos para FPGAs de baixo custo, mas o fator de decaimento $\bar{A}_t$ opera próximo de 1 e é frágil sob aritmética apenas com deslocamentos. Apresentamos o **BitMamba-CV**, um co-projeto de Mamba-2 ternário (1,58 bit) e de um* datapath *de* scan *seletivo sem multiplicador no decaimento para a Intel Cyclone V. Nós (i) treinamos projeções ternárias no domínio de Hadamard (`H-BitLinear`) junto com um* scan *compatível com aritmética inteira (escalas estáticas em potência de dois e Δ_t em potência de dois); (ii) propomos a discretização* Complementary Power-of-Two *(CPoT, $\{2^{-k}, 1-2^{-k}\}$) de $\bar{A}_t$, implementada com um deslocamento e uma subtração; e (iii) mostramos que o CPoT só é correto com precisão de estado suficiente: um estado de 16 bits com truncamento congela os canais de memória longa (SQNR do* datapath *inteiro de 37,0 dB), enquanto um estado de 32 bits com 10 bits de guarda e arredondamento atinge 99,0 dB, equivalente a um multiplicador Q1.15 (99,3 dB) com menos blocos DSP; numa tarefa de recordação com $L = 256$, o mesmo modelo treinado cai de 100,0% para 85,6% com o estado de 16 bits.*

Modelos ternários com scan *inteiro mantêm a acurácia FP32 no ECG5000 e na recordação com $L = 256$, mas o treinamento ternário não converge na maioria das sementes com $L = 1024$. Todos os módulos RTL foram verificados bit a bit contra um modelo de referência inteiro (9.384 vetores, 0 divergências) e caracterizados com síntese de código aberto para Cyclone V. Fechamento de* timing*, latência em placa e potência ficam para a fase de hardware e não são reivindicados aqui.*

1 Introdução

A inferência de modelos de sequência em FPGAs de baixo custo, como a família Intel Cyclone V (28 nm, sem HBM), é limitada por memória *on-chip* e por blocos DSP escassos [7]. Em Transformers, o *KV-cache* cresce linearmente com o comprimento L da sequência; para um modelo de 4 camadas com $D = 128$ e chaves/valores em INT8, são $2 \cdot 4 \cdot 128 \cdot L$ bytes, isto é, 1 KiB por token. Modelos de espaço de estados seletivos (Mamba-1 [6] e Mamba-2 [4]) substituem a atenção por uma recorrência linear cujo estado tem tamanho constante em L, o que elimina esse crescimento.

Dois obstáculos aparecem ao mapear o Mamba-2 em uma FPGA de borda. Primeiro, as projeções lineares (`in_proj` e `out_proj`) concentram a maior parte dos parâmetros (96,8% no modelo usado aqui) e, em INT8, consomem DSPs. Segundo, o decaimento $\bar{A}_t = \exp(-\Delta_t A_h)$ dos canais de memória longa fica em $[0{,}90;\ 0{,}999]$, onde quantizações grosseiras alteram a constante de tempo do estado.

Aceleradores Mamba em FPGA já usam quantização de 4 e 8 bits em FPGAs de grande porte; o *LightMamba*, por exemplo, usa rotação e restringe os fatores de *re-escala* da quantização do SSM a potências de dois [15, 12]. Aceleradores ternários (1,58 bit) em FPGA, como *TerEffic* e *TeLLMe* [17, 10], visam Transformers. Trabalhos algorítmicos mostram que o Mamba ternário funciona em GPU [18, 5, 11], mas mantêm o SSM em ponto flutuante.

Contribuições. (1) Um modelo Mamba-2 com projeções ternárias `H-BitLinear` [13] e um *scan* treinado já com as restrições do hardware inteiro: escalas estáticas em potência de dois para x_t, B_t, C_t e passo $\Delta_t = 2^{-j}$. (2) A discretização CPoT de $\bar{A}_t$, que usa um deslocamento e uma subtração, comparada com PoT simples e com um multiplicador Q1.15. (3) Uma análise da precisão do estado inteiro, mostrando que a implementação ingênua (16 bits, truncamento) congela a memória longa, e a correção com bits de guarda e arredondamento. (4) RTL parametrizável (motor matriz-vetor INT8, ternário 2-bit e Radix-3; FWHT; *lane* de *scan*) verificado bit a bit e caracterizado em recursos para Cyclone V.

Escopo. Este artigo não reporta frequência máxima, latência em placa nem potência: esses dados dependem da síntese no Intel Quartus e da execução na placa DE10-Nano, que ainda não foram realizadas (Seção 7).

2 Fundamentação

2.1 Mamba-2

O Mamba-2 [4] divide o estado em H cabeças e usa um decaimento escalar $A_h > 0$ por cabeça. Para cada canal p da cabeça h e passo t:

$$\bar{A}_t = \exp(-\Delta_t A_h), \qquad \mathbf{h}_t = \bar{A}_t\, \mathbf{h}_{t-1} + \Delta_t\, x_t\, \mathbf{B}_t, \qquad y_t = \mathbf{C}_t^\top \mathbf{h}_t + D_h x_t, \quad (1)$$

com $\mathbf{h}_t, \mathbf{B}_t, \mathbf{C}_t \in \mathbb{R}^N$. Δ_t, $\mathbf{B}_t$ e $\mathbf{C}_t$ dependem da entrada (seletividade). A constante de tempo do canal é $\tau = -1/\ln \bar{A}_t$; para $\bar{A}_t = 1 - 2^{-k}$, $\tau \approx 2^k$ passos.

2.2 `H-BitLinear`: Hadamard sem perder a ternaridade

Rotações de Hadamard pós-treinamento [2] usam $\mathbf{W}\mathbf{x} = (\mathbf{W}\mathbf{H}^\top)(\mathbf{H}\mathbf{x})$; com $\mathbf{W}$ ternária, $\mathbf{W}\mathbf{H}^\top$ deixa de ser ternária. Seguindo o BitNet v2 [13], aplicamos a FWHT em blocos de 16 dentro do grafo de treinamento e aprendemos $\tilde{\mathbf{W}}$ diretamente no domínio transformado:

$$\tilde{\mathbf{x}} = Q_8\big(\tfrac{1}{4}\mathbf{H}_{16}\,\mathrm{RMSNorm}(\mathbf{x})\big)\,, \qquad \tilde{\mathbf{W}} = \mathrm{round}(\mathrm{clip}(\mathbf{W}/\gamma, -1, 1))\,, \quad \gamma = \overline{|\mathbf{W}|}, \quad (2)$$

com Q_8 a quantização simétrica INT8 por token e estimador *straight-through*. Em hardware, $\mathbf{H}_{16}$ é calculada exatamente com somas e subtrações; o fator $1/4$ é absorvido na escala de requantização.

2.3 Discretização CPoT de $\bar{A}_t$

Com $\bar{A}_t = 2^{-k}$ (PoT simples) a multiplicação vira deslocamento, mas não há nível entre 0,5 e 1: com $k \geq 1$ todo canal lento é levado a $\leq 0{,}5$ ($\tau \leq 1{,}4$ passos); com $k \geq 0$ ele é levado a 1 e deixa de esquecer. Como $\exp(-\Delta_t A_h) \approx 1 - \Delta_t A_h$ para $\Delta_t A_h$ pequeno, aproximamos o *complemento* por uma potência de dois:

$$\bar{A}^{\mathrm{CPoT}}(m,k) = \begin{cases} 2^{-k}, & m = 0 \;\; (\bar{A}_t \leq 0{,}625) \\ 1 - 2^{-k}, & m = 1 \;\; (\bar{A}_t > 0{,}625) \end{cases} \qquad \bar{A}^{\mathrm{CPoT}} h = \begin{cases} h \gg k \\ h - (h \gg k) \end{cases}, \quad (3)$$

com $k \in \{1, \ldots, 15\}$ escolhido pelo arredondamento no domínio logarítmico de cada ramo. A variante CPoT-2 ($1 - 2^{-k_1} \mp 2^{-k_2}$) foi avaliada apenas em software. Como $\Delta_t = 2^{-j}$ e A_h é fixo após o treinamento, o código (m, k) de cada cabeça é uma tabela de 16 entradas indexada por j: o hardware não precisa calcular $\exp$.

2.4 Precisão necessária no estado

Com estado inteiro e deslocamento truncado, $h - (h \gg k) = h$ sempre que $0 \leq h < 2^k$: o decaimento para e o canal congela num valor positivo. Para $h < 0$, $h \gg k = -1$ e o estado sobe de 1 em 1 até zero, criando uma assimetria de sinal. Com k até 12, um estado Q8.8 de 16 bits congela valores de magnitude relevante. A correção usa (i) G bits de guarda (o estado guarda $h \cdot 2^G$), (ii) arredondamento $s = (h + 2^{k-1}) \gg k$ e (iii) estado de 32 bits com saturação. A injeção $\Delta_t x_t B_t$ vira $((x_t B_t) \ll G) \gg j$ e a leitura $y_t = (\sum_n C_t[n]\, h_t[n]) \gg G$.

3 Trabalhos Relacionados

A Tabela 1 resume os trabalhos mais próximos. Componentes isolados já existem: empacotamento de 5 trits por byte em *TerEffic* [17], multiplicação por tabela em T-MAC/*TeLLMe* [14, 10] e Hadamard no QAT em BitNet v2 [13]. Não encontramos trabalho que una projeções ternárias a um *scan* inteiro do Mamba-2 com decaimento sem multiplicador e análise da precisão do estado.

Tabela 1: Trabalhos relacionados.

Trabalho	**Modelo**	**Pesos/Ativ.**	**SSM**	**Plataforma**	**Diferença para este trabalho**
BitNet b1.58 [9]	Transformer	1,58b / INT8	—	GPU	Sem SSM; KV-cache $O(L)$.
BitNet v2 [13]	Transformer	1,58b / INT4	—	GPU	Origem do `H-BitLinear`; sem hardware.
Quamba/Quamba2 [2, 3]	Mamba-1/2	W8A8 / W4A8	INT8	GPU	PTQ; rotação absorvida nos pesos.
Bi-Mamba [11]	Mamba-1	1b / FP16	FP16	GPU	Pesos binários; SSM em ponto flutuante.
Slender-Mamba [18]	Mamba-2	1,58b / INT8	FP16	GPU	Sem *datapath* inteiro do *scan*.
Ternary Mamba [5]	Mamba-2	1,58b / FP16	FP16	GPU	QAT agrupado; SSM em FP16.
MARCA [8]	Mamba-1	INT8 / FP16	PWL	ASIC (sim.)	Multiplicadores densos.
LightMamba [15]	Mamba-2	W4A4	PoT nas escalas	Alveo U280, VCK190	FPGA de grande porte; DSPs nas projeções.
FastMamba [12]	Mamba-2	W8A8	PoT nas escalas	Virtex-7	Pesos lineares em 8 bits.
TerEffic [17]	Transformer	1,6b / INT8	—	Alveo U280	KV-cache $O(L)$, HBM.
TeLLMe [10]	Transformer	1,58b / INT8	—	Kria KV260	KV-cache $O(L)$.
BitMamba-CV	**Mamba-2**	**1,58b / INT8**	**CPoT inteiro**	**Cyclone V (RTL)**	—

4 Arquitetura

O RTL implementado (Verilog-2005, diretório `hw/rtl`) contém quatro blocos, todos parametrizáveis.

Motor matriz-vetor (`bitlinear_engine.v`). Calcula $\mathbf{y} = \mathbf{W}\mathbf{x}$ com $\mathbf{W} \in \mathbb{Z}^{M\times K}$ e P vias paralelas: a cada ciclo lê uma palavra de pesos da RAM interna, forma P produtos e soma-os numa árvore binária com um registrador por nível (latência $\log_2 P$); um acumulador soma as K/P fatias de cada linha. A latência medida em simulação é $MK/P + \log_2 P + 3$ ciclos por produto matriz-vetor. Há três codificações de peso: **INT8** (produtos em DSP), **ternário 2-bit** (código {não-zero, negativo}; o produto vira um multiplexador $\{+x, 0, -x\}$ em lógica) e **Radix-3**, em que 5 trits ocupam 1 byte ($\sum_i (w_i + 1)3^i \leq 242$). No Radix-3, 5 linhas consecutivas de P pesos ($5P$ trits) ocupam exatamente uma palavra de $8P$ bits, o que dá 1,6 bit/peso; P decodificadores combinacionais (tabela de 243 entradas) expandem a palavra e um multiplexador 5:1 seleciona a linha corrente.

FWHT (`fwht16.v`). Transformada de Walsh-Hadamard de 16 pontos exata, com 4 estágios registrados de soma/subtração, entrada INT8 e saída de 12 bits sem saturação.

Lane* de *scan (`scan_lane.v`). Implementa a Eq. 1 para um canal e N estados em paralelo, com os códigos (m, k, j) chegando junto com cada token. Estágios: produto $B_t x_t$ (DSP), recorrência com laço de 1 ciclo (decaimento CPoT da Seção 2.4, injeção por deslocamento, saturação), produtos $C_t h_t$ (DSP) e árvore de soma. Parâmetros selecionam a largura do estado, os bits de guarda, o arredondamento e o tipo de decaimento (CPoT ou multiplicador Q1.15, usado como baseline).

Não implementado em RTL: RMSNorm, quantização dinâmica por token, `Conv1D`, SiLU, o termo $D_h x_t$ e a integração com o processador ARM (HPS) via bar-

ramento AXI. No modelo de software, *embedding*, `Conv1D` e a camada de classificação permanecem em ponto flutuante.

5 Metodologia

5.1 Especificação única e fluxo de verificação

O modelo de referência (`sw/models/golden_model_bitexact.py`) define, em aritmética inteira, cada bloco do RTL: decodificação Radix-3, FWHT, produto matriz-vetor e o passo do *scan* (larguras, saturações e deslocamentos aritméticos). O script `sw/verify_rtl.py` gera os estímulos e as saídas esperadas a partir desse modelo, simula cada módulo no Icarus Verilog 12 e exige zero divergências. Foram cobertos: os 256 códigos do decodificador (exaustivo); 2.000 vetores da FWHT, incluindo os extremos ± 128; o motor matriz-vetor nas três codificações com $P \in \{16, 32, 64\}$, pesos ternários com 30% de zeros e ativações INT8 uniformes; e três configurações do *scan* com 4 sequências de 512 tokens, decaimentos majoritariamente em $1 - 10^{-[0,2;\,3,5]}$, 15% de tokens de decaimento rápido e j aleatório. Para confirmar que os testes detectam erros, simulamos mutantes (arredondamento desligado; codificação de peso trocada), que produziram 780/2.048 e 160/160 divergências. A emulação inteira usada no treinamento (Seção 5.2) é checada contra o mesmo modelo de referência.

5.2 Treinamento QAT e ablação

Modelo. Classificador com *embedding* linear, 2 blocos Mamba-2 ($D = 64$, expansão 2, $H = 4$ cabeças, $N = 16$, `Conv1D` com $k = 4$), RMSNorm e camada linear aplicada ao último token; 68.253 parâmetros, dos quais 66.048 nas projeções. A inicialização segue o Mamba-2: $A_h \sim \mathcal{U}[1, 16]$ e $\Delta \sim \log \mathcal{U}[10^{-3}, 10^{-1}]$.

Configurações. A0: FP32. A1: W8A8 (pesos INT8 por linha, ativações INT8 por token). A2: pesos ternários sem FWHT. A3: `H-BitLinear`. De A4 a A8 o *scan* usa as restrições do hardware: x_t, B_t, C_t em INT8 com escala estática 2e calibrada por média móvel do máximo absoluto, $\Delta_t = 2^{-j}$ com $j \in [0, 15]$, e $\bar{A}_t = \exp(-\Delta_t A_h)$ quantizado por Q1.15 (A4), PoT com $k \geq 1$ (A5), PoT com $k \geq 0$ (A6), CPoT (A7) ou CPoT-2 (A8). Em A1–A3 o *scan* usa quantização dinâmica por token e $\bar{A}_t$ exato. Todas as quantizações usam estimador *straight-through*.

Emulação inteira. Para A4 e A7, o modelo treinado também é avaliado substituindo o *scan* em ponto flutuante pela recorrência inteira do modelo de referência (A4†: multiplicador Q1.15, 32 bits; A7†: CPoT, 32 bits, 10 bits de guarda, arredondamento; A7‡: CPoT com o estado legado de 16 bits e truncamento). Isso mede o efeito da aritmética do RTL na acurácia sem retreinar.

Tarefas. (i) **ECG5000** [1]: 5 classes, $L = 140$, divisão oficial com 500 séries de treino e 4.500 de teste; como as classes são desbalanceadas, reportamos também a acurácia balanceada (média das revocações por classe). (ii) **Recordação atrasada** (sintética): 4 classes; a classe aparece apenas nos 8 primeiros passos (*one-hot* de amplitude 2 em 4 canais) e o restante da sequência é ruído gaussiano ($\sigma = 0{,}1$, 8 canais). A saída é lida em $t = L - 1$, então a tarefa exige memória de $\approx L$ passos. Usamos $L = 256$ (4.000 treino, 1.000 teste) e $L = 1024$ (2.000/500). Numa versão preliminar com canais de *ou-*

tliers ($\sigma = 12$) nem o modelo FP32 aprendeu a tarefa para $L \geq 128$, e esses canais foram removidos.

Treinamento. AdamW (taxa máxima $2 \cdot 10^{-3}$, *one-cycle*, *weight decay* 10^{-2}), recorte de gradiente em 1, entropia cruzada; 40 épocas (ECG5000) e 20 épocas (recordação); 5 sementes para ECG5000 e $L = 256$, 3 para $L = 1024$. Sem seleção de modelo no conjunto de teste: avalia-se o modelo da última época. Reportamos média $\pm$ desvio padrão entre sementes.

5.3 Experimentos numéricos de memória longa

Sem treinamento (`sw/long_range_experiment.py`): **E1** mede o erro de cada esquema sobre 2.000 valores de $\bar{A} \in [0{,}90;\ 0{,}999]$, tanto em $\bar{A}$ quanto na constante de tempo τ. **E2** mede a fidelidade da leitura final y_L com L de 64 a 16.384, 8 cabeças com τ log-espaçado de 16 até L, um gatilho forte em $t = 0$ e ruído; só $\bar{A}$ é quantizado e o estado é float (5 sementes). **E3** é a resposta ao impulso do estado inteiro do RTL sob $1-2^{-k}$. **E4** é o SQNR da saída do *datapath* inteiro com entradas INT8 aleatórias, $k \in [4, 12]$, $j = k-1$ e $T = 4.096$, contra a recorrência em float com os *mesmos* códigos quantizados: mede apenas o erro aritmético do *datapath*.

5.4 Estimativa de recursos

Sem o Intel Quartus disponível, sintetizamos cada variante com o Yosys 0.33 [16] (`synth_intel_alm -family cyclonev`). O passo ABC9 dessa versão aborta nesses projetos (asserção interna), então o fluxo roda até o mapeamento de LUTs e usa o ABC clássico (`abc -lut 6`). Reportamos células mapeadas (LUTs de até 6 entradas, células de *carry*, *flip-flops*, blocos M10K e multiplicadores 9×9/18×18/27×27), não ALMs do *fitter*. O número de blocos DSP é estimado pela capacidade de um bloco Cyclone V (3×9×9, 2×18×18 ou 1×27×27). Frequência máxima não é estimada. O motor foi sintetizado para uma projeção com $K = 256$ e $M = 512$ (131.072 pesos).

5.5 Reprodutibilidade

Todos os números deste artigo são gerados por `sw/make_paper_assets.py` a partir dos arquivos em `sw/results/`, produzidos por `verify_rtl.py`, `long_range_experiment.py`, `train_ablation.py` e `synth_yosys.py`. O script `synth_quartus.py` (síntese, *fit* e *timing* no Quartus Prime Lite para o dispositivo `5CSEBA6U23I7`) está pronto, mas ainda não foi executado.

6 Resultados

6.1 Discretização de $\bar{A}_t$ e precisão do estado

A Tabela 2 e a Figura 1(a) mostram o comportamento esperado do PoT simples. Com $k \geq 1$, todos os valores da faixa viram $0{,}5$: o erro mediano de τ é 92,5% e o SQNR fica em 0 dB. Com $k \geq 0$, todos viram 1 e o estado deixa de esquecer: a direção da leitura dos canais lentos é preservada, mas sua magnitude diverge (SQNR negativo). O CPoT usa 8 níveis na faixa com erro mediano de τ de 20,5%; o CPoT-2 reduz esse erro para 2,4%. O Q1.15 com multiplicador continua sendo o mais preciso.

Tabela 2: E1 e E2: quantização de $\bar{A} \in [0{,}90;\ 0{,}999]$ (níveis distintos usados, erro absoluto máximo, erro relativo mediano de τ) e SQNR (dB) da leitura y_L com estado em float (média de 5 sementes).

Esquema	**Hardware**	**Níveis**	**máx** $\lvert\Delta\bar{A}\rvert$	**Erro** τ	$L = 256$	$L = 4.096$	$L = 16.384$
Q1.15 (mult.)	1 mult.	2000	0,0000	0,0%	81,3	68,8	52,9
PoT 2^{-k}, $k \geq 1$	1 shift	1	0,4990	92,5%	0,0	0,0	0,0
PoT 2^{-k}, $k \geq 0$	1 shift	1	0,1000	∞	-15,9	-17,4	-15,7
CPoT	1 shift + 1 sub.	8	0,0366	20,5%	20,7	22,3	37,5
CPoT-2 (só SW)	2 shifts + 2 sub.	92	0,0092	2,4%	50,3	51,1	41,3

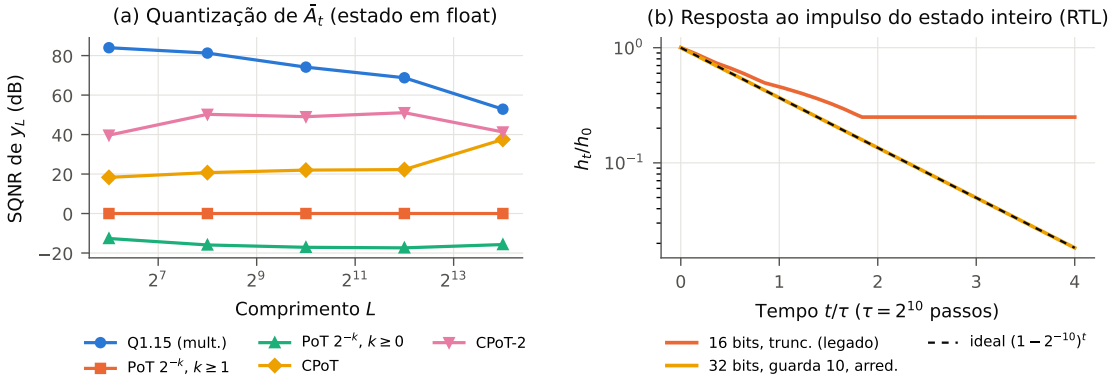


Figura 1: (a) E2: SQNR da leitura y_L em função de L (estado em float). (b) E3: resposta ao impulso do estado inteiro do RTL com $\bar{A} = 1 - 2^{-10}$. O estado legado de 16 bits congela pouco antes de 2τ; o estado corrigido segue a curva ideal.

A Tabela 3 confirma a análise da Seção 2.4. Com o estado legado (16 bits, truncamento), após 4τ o estado vale 54,6× o ideal para $k = 12$ (e 3,43× já para $k = 8$), e o SQNR do *datapath* cai para 37,0 dB. Com 32 bits, 10 bits de guarda e arredondamento, a resposta ao impulso coincide com a ideal e o SQNR sobe para 99,0 dB, no mesmo nível do baseline com multiplicador (99,3 dB).

6.2 Ablação QAT

A Tabela 4 leva a cinco observações.

ECG5000. Todas as configurações ficam a menos de 0,5 ponto percentual da re-

Tabela 3: E3 e E4: estado inteiro do RTL. Razão h_t/h_t^{ideal} após 4τ passos de um impulso ($\bar{A} = 1 - 2^{-k}$) e SQNR do *datapath* inteiro com entradas aleatórias (3 sementes).

Estado	$k = 8$	$k = 10$	$k = 12$	**SQNR E4 (dB)**
Legado: 16 bits, sem guarda, truncamento	3,43	13,66	54,61	37,0
Corrigido: 32 bits, 10 bits de guarda, arredondamento	1,00	1,00	1,00	99,0
Baseline: multiplicador Q1.15, 32 bits	—	—	—	99,3

Tabela 4: Acurácia de teste (%, média ± desvio padrão entre sementes). †/‡: o mesmo modelo treinado, avaliado com o *scan* inteiro bit-exato do RTL.

ID	Configuração	Pesos	FWHT	$\bar{A}_t$	ECG5000 ($L = 140$) Acur.	ECG5000 ($L = 140$) Acur. bal.	Cue $L = 256$ Acur.	Cue $L = 1024$ Acur.
A0	FP32 (referência)	FP32	—	float exato	$92{,}7 \pm 0{,}3$	$52{,}0 \pm 0{,}6$	$100{,}0 \pm 0{,}0$	$100{,}0 \pm 0{,}0$
A1	W8A8	INT8	não	float exato	$92{,}7 \pm 0{,}3$	$53{,}0 \pm 1{,}0$	$100{,}0 \pm 0{,}0$	$100{,}0 \pm 0{,}0$
A2	Ternário	1,58b	não	float exato	$93{,}1 \pm 0{,}1$	$53{,}4 \pm 1{,}8$	$100{,}0 \pm 0{,}0$	$50{,}0 \pm 35{,}4$
A3	+ H-BitLinear	1,58b	sim	float exato	$92{,}5 \pm 0{,}1$	$52{,}5 \pm 1{,}4$	$100{,}0 \pm 0{,}0$	$49{,}3 \pm 35{,}8$
A4	+ *scan* HW	1,58b	sim	Q1.15 (mult.)	$92{,}7 \pm 0{,}4$	$52{,}6 \pm 0{,}6$	$100{,}0 \pm 0{,}0$	$49{,}1 \pm 36{,}0$
A5	+ *scan* HW	1,58b	sim	PoT, $k \geq 1$	$91{,}7 \pm 0{,}2$	$50{,}7 \pm 1{,}0$	$24{,}1 \pm 1{,}2$	$21{,}9 \pm 0{,}4$
A6	+ *scan* HW	1,58b	sim	PoT, $k \geq 0$	$92{,}5 \pm 0{,}4$	$52{,}3 \pm 0{,}8$	$100{,}0 \pm 0{,}0$	$100{,}0 \pm 0{,}0$
A7	**+ *scan* HW**	1,58b	sim	**CPoT**	$92{,}8 \pm 0{,}3$	$52{,}9 \pm 1{,}3$	$100{,}0 \pm 0{,}0$	$39{,}0 \pm 21{,}2$
A8	+ *scan* HW	1,58b	sim	CPoT-2 (só SW)	$92{,}7 \pm 0{,}1$	$52{,}8 \pm 0{,}7$	$100{,}0 \pm 0{,}0$	$39{,}4 \pm 22{,}8$
A4†	A4 avaliado com *scan* inteiro bit-exato			Q1.15, 32b	$92{,}7 \pm 0{,}4$	$52{,}6 \pm 0{,}6$	$100{,}0 \pm 0{,}0$	$49{,}2 \pm 35{,}9$
A7†	**A7 avaliado com *scan* inteiro bit-exato**			**CPoT, 32b**	$92{,}8 \pm 0{,}3$	$52{,}9 \pm 1{,}3$	$100{,}0 \pm 0{,}0$	$39{,}3 \pm 21{,}2$
A7‡	A7 com estado legado (16b, trunc.)			CPoT, 16b	$92{,}8 \pm 0{,}4$	$52{,}9 \pm 1{,}9$	$85{,}6 \pm 11{,}8$	$24{,}1 \pm 0{,}7$

ferência FP32 (92,7%), exceto o PoT com $k \geq 1$ (91,7%); o CPoT com *scan* de hardware obtém 92,8%. A acurácia balanceada fica perto de 52% em todas as configurações, inclusive FP32: com 500 séries de treino, as classes minoritárias não são aprendidas, o que é uma limitação do modelo e do protocolo, não da quantização. Como $L = 140$, o estado legado de 16 bits não chega a prejudicar (92,8%).

Recordação com $L = 256$. Todas as configurações resolvem a tarefa (acurácia acima de 90% em 5/5 sementes no CPoT), exceto o PoT com $k \geq 1$ (24,1%, nível de acaso), que não consegue representar $\bar{A}_t > 0{,}5$.

Recordação com $L = 1024$. FP32, W8A8 e PoT com $k \geq 0$ resolvem todas as sementes. Os modelos ternários, *com qualquer* decaimento (A2, A3, A4, A7, A8), resolvem no máximo uma de três sementes (acurácia acima de 90% em 1/3 sementes em A3 e em 0/3 em A7, cuja melhor semente acerta parcialmente); nas demais sementes eles ficam no nível de acaso. O gargalo nesse comprimento é a otimização dos pesos ternários dentro do orçamento usado (20 épocas, 2.000 sequências), e não a discretização de $\bar{A}_t$; com 3 sementes, os desvios são grandes e as médias de A2–A8 não permitem ordenar os esquemas.

CPoT × PoT com $k \geq 0$. Nessas tarefas o PoT com $k \geq 0$ empata com o CPoT ($L = 256$) ou o supera ($L = 1024$). A tarefa de recordação só exige reter informação ($\bar{A}_t \to 1$), e o Δ_t seletivo permite zerar o estado quando necessário; ela não exige constantes de tempo intermediárias, que é onde o CPoT se diferencia (E1 e E2). Mostrar essa vantagem em acurácia exige uma tarefa com esquecimento gradual e ainda está em aberto. A FWHT (A2 → A3) também não mostrou efeito mensurável nessas tarefas, que não têm canais de *outlier*.

Aritmética inteira do RTL. Avaliar o modelo treinado com o *scan* inteiro bit-exato reproduz a acurácia em ponto flutuante (A4† e A7†, diferenças de no máximo alguns décimos). Com o estado legado (A7‡), o mesmo modelo cai de 100,0% para 85,6% em $L = 256$ e de 39,3% para 24,1% em $L = 1024$: o problema de precisão da Seção 2.4 também aparece numa tarefa treinada.

Nos modelos ternários treinados, 26,6% dos pesos das projeções valem zero (mé-

dia de A7). As projeções ocupam 129,0 KiB em FP16, 64,5 KiB em INT8, 16,1 KiB em 2 bits e 12,9 KiB em Radix-3.

6.3 Verificação do RTL

Tabela 5: Verificação RTL × modelo de referência inteiro (Icarus Verilog 12).

Módulo	**Configurações**	**Saídas comparadas**	**Divergências**
Decodificador Radix-3 (exaustivo, 256 códigos)	1	256	0
FWHT 16 pontos (vetores aleatórios + extremos)	1	2.000	0
Motor matriz-vetor (INT8/2-bit/Radix-3, $P \in \{16, 32, 64\}$)	9	984	0
Lane de *scan* (CPoT 32b, CPoT 16b legado, Q1.15)	3	6.144	0
Total	14	**9.384**	**0**

Todos os 14 testes da Tabela 5 terminam com zero divergências. Nenhum teste foi executado em hardware.

6.4 Recursos estimados (Yosys, Cyclone V)

Tabela 6: Motor matriz-vetor ($K = 256$, $M = 512$): células mapeadas pelo Yosys para Cyclone V. Ciclos: $MK/P + \log_2 P + 3$, confirmado em simulação. Sem ALMs/$F_{\max}$ do Quartus.

Pesos	P	**bit/peso**	**LUTs**	***Carry***	**FFs**	**M10K**	**Mult. 9×9**	**Ciclos**
INT8 (DSP)	16	8,00	210	360	844	128	16	8.199
	32	8,00	352	685	1.612	128	32	4.104
	64	8,00	893	1.338	3.148	103	64	2.057
Ternário 2-bit	16	2,00	370	304	732	32	0	8.199
	32	2,00	672	573	1.388	32	0	4.104
	64	2,00	1.533	1.114	2.700	26	0	2.057
Ternário Radix-3	16	1,60	1.019	305	736	26	0	8.199
	32	1,60	1.972	574	1.392	26	0	4.104
	64	1,60	4.101	1.115	2.704	26	0	2.057

Projeções (Tabela 6). Os motores ternários não usam multiplicadores, enquanto o INT8 usa um 9×9 por via (16 com $P = 16$). Com $P = 16$, a memória de pesos cai de 128 blocos M10K (INT8) para 32 (2-bit) e 26 (Radix-3). O Radix-3 custa LUTs: 1.019 contra 370 no 2-bit com $P = 16$, porque cada via tem um decodificador (38 LUTs) e um multiplexador 5:1. Com $P = 64$ a vantagem em M10K desaparece (26 contra 26 blocos): a largura da palavra, e não o total de bits, define o número de blocos, enquanto o custo em LUTs sobe para 4.101 contra 1.533. A economia de 20% em bits do Radix-3 só se traduz em blocos quando a geometria da memória permite, ou em banda quando os pesos vêm da DRAM, regime que não foi medido.

***Scan* (Tabela 7).** O decaimento CPoT elimina os multiplicadores $h \cdot \bar{A}$: a *lane* proposta usa 27 blocos DSP estimados contra 54 do baseline Q1.15, ao custo de mais

Tabela 7: Blocos do *scan* ($N = 16$ estados) e auxiliares: células mapeadas pelo Yosys para Cyclone V e blocos DSP estimados.

Bloco	**LUTs**	***Carry***	**FFs**	**9×9**	**18×18**	**27×27**	**DSP est.**
CPoT, 32b + guarda (proposto)	3.101	4.144	2.332	32	0	16	27
CPoT, 16b (legado, incorreto)	1.675	1.694	1.590	16	16	0	14
Multiplicador Q1.15, 32b	2.483	3.402	2.343	64	0	32	54
FWHT 16 pontos	0	720	724	0	0	0	0
Decodificador Radix-3 (1 byte)	38	0	0	0	0	0	0

lógica (3.101 contra 2.483 LUTs, devido ao deslocador de 32 bits). Os DSPs restantes vêm de $B_t x_t$ e $C_t h_t$, que continuam sendo multiplicações. A Cyclone V `5CSEBA6U23I7` tem 112 blocos DSP, então o número de *lanes* paralelas é limitado por DSP no baseline e por esses produtos no CPoT.

7 Limitações e Próximos Passos

- **Sem Quartus e sem placa.** Os recursos são células do Yosys, não ALMs do *fitter*, e não há F_{max}, *throughput*, latência nem potência. A próxima etapa é rodar `synth_quartus.py` e medir na DE10-Nano (ARM via ponte AXI, contadores de ciclo, medidor de corrente na alimentação).
- **Integração incompleta.** RMSNorm, quantização dinâmica, `Conv1D`, SiLU e o controle do bloco não estão em RTL. No QAT, a FWHT recebe a ativação em ponto flutuante; no hardware, ela receberá INT8 já quantizado.
- **Escala.** Modelos pequenos (≈68.253 parâmetros) e tarefas de classificação; a tarefa de recordação é sintética. Modelos de linguagem e o regime com pesos na DRAM (*memory-bound*) não foram avaliados.
- **Alternativas de motor.** Multiplicação por tabela (TLMM) e salto de pesos nulos não foram implementados nesta versão.

8 Conclusão

Apresentamos o BitMamba-CV, um co-projeto de Mamba-2 ternário e de um *datapath* de *scan* para a Intel Cyclone V, com especificação inteira única, RTL verificado bit a bit (9.384 saídas, 0 divergências) e todos os números gerados a partir dos resultados. Três conclusões se sustentam com os dados atuais. (i) A discretização CPoT representa constantes de tempo longas com um deslocamento e uma subtração (erro mediano de τ de 20,5% em $[0{,}90;\ 0{,}999]$, contra 92,5% do PoT com $k \geq 1$) e reduz os blocos DSP estimados da *lane* de *scan* de 54 para 27. (ii) Essa discretização só é correta com estado de 32 bits, bits de guarda e arredondamento: o estado de 16 bits truncado congela a memória longa, derruba o SQNR do *datapath* de 99,0 para 37,0 dB e a acurácia da tarefa de recordação com $L = 256$ de 100,0% para 85,6%. (iii) O modelo ternário com *scan* inteiro mantém a acurácia FP32 no ECG5000 e na recordação com $L = 256$; com $L =$

1024, o treinamento ternário não converge na maioria das sementes. Ficam em aberto: a vantagem do CPoT sobre o PoT com $k \geq 0$ em acurácia, a síntese no Quartus (ALMs e $F_{\max}$) e a validação na placa DE10-Nano (latência e potência).

Referências

[1] Yanping Chen, Eamonn Keogh, Bing Hu, Nurjahan Begum, Anthony Bagnall, Abdullah Mueen, and Gustavo Batista. The UCR time series classification archive. `https://www.cs.ucr.edu/~eamonn/time_series_data/` (dataset ECG5000), 2015.

[2] Hung-Yueh Chiang, Chi-Chih Chang, Natalia Frumkin, and Diana Marculescu. Quamba: A post-training quantization recipe for selective state space models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.13889.

[3] Hung-Yueh Chiang, Chi-Chih Chang, and Diana Marculescu. Quamba2: A robust and scalable post-training quantization framework for selective state space models. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.22879.

[4] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2405.21060.

[5] Ramprasath Ganesaraja, Sahil Dilip Panse, and Swathika N. Ternary Mamba: Grouped quantization-aware training of W1.58A16 state space models. *arXiv preprint arXiv:2606.18114*, 2026.

[6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2312.00752.

[7] Intel Corporation (Altera). *Cyclone V Device Handbook: Volume 1 – Device Interfaces and Integration*, 2023.

[8] Jinhao Li, Shan Huang, Jiaming Xu, Jun Liu, Li Ding, Ningyi Xu, and Guohao Dai. MARCA: Mamba accelerator with ReConfigurable architecture. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.

[9] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

[10] Ye Qiao et al. TeLLMe: An energy-efficient ternary LLM accelerator for prefilling and decoding on edge FPGAs. *arXiv preprint arXiv:2504.16266*, 2025.

[11] Shengkun Tang, Liqun Ma, Haonan Li, Mingjie Sun, and Zhiqiang Shen. Bi-Mamba: Towards accurate 1-bit state space models. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2411.11843.

[12] Aotao Wang, Haikuo Shao, Shaobo Ma, and Zhongfeng Wang. FastMamba: A high-speed and efficient Mamba accelerator on FPGA with accurate quantization. *arXiv preprint arXiv:2505.18975*, 2025.

[13] Hongyu Wang, Shuming Ma, and Furu Wei. BitNet v2: Native 4-bit activations with Hadamard transformation for 1-bit LLMs. *arXiv preprint arXiv:2504.18415*, 2025.

[14] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge. In *European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2407.00088.

[15] Renjie Wei, Songqiang Xu, Linfeng Zhong, Zebin Yang, Qingyu Guo, Yuan Wang, Runsheng Wang, and Meng Li. LightMamba: Efficient Mamba acceleration on FPGA with quantization and hardware co-design. In *Design, Automation and Test in Europe (DATE)*, 2025. arXiv:2502.15260.

[16] Claire Wolf. Yosys open synthesis suite. `https://yosyshq.net/yosys/`, 2023.

[17] Chenyang Yin, Zhenyu Bai, Pranav Venkatram, Shivam Aggarwal, Zhaoying Li, and Tulika Mitra. TerEffic: Highly efficient ternary LLM inference on FPGA. *arXiv preprint arXiv:2502.16473*, 2025.

[18] Zhenxuan Yu, Takeshi Kojima, Yutaka Matsuo, and Yusuke Iwasawa. SlenderMamba: Fully quantized Mamba in 1.58 bits from head to toe. In *International Conference on Computational Linguistics (COLING)*, 2025.